Java Core Concepts, Microservices, and Spring - Detailed Answers

Java Core Concepts

1. Lambda Expressions

Lambda expressions are anonymous functions that provide a concise way to implement functional interfaces (interfaces with a single abstract method).

Key Features:

- No need to specify return type
- No need to specify parameter types (type inference)
- Can be used as method arguments

Example:

```
java
Copy
// Traditional way with anonymous class
Runnable r1 = new Runnable() {
    public void run() {
        System.out.println("Hello");
    }
};

// With lambda expression
Runnable r2 = () -> System.out.println("Hello");

// Another example with Comparator
Comparator<String> comp = (s1, s2) -> s1.compareTo(s2);

// Lambda with parameters and return
MathOperation add = (a, b) -> a + b;
System.out.println(add.operate(5, 3)); // 8
```

2. Java 8 Features

Major Java 8 features include:

1. **Lambda Expressions** - Functional programming
2. **Stream API** - Bulk data operations
3. **Optional Class** - Null-safe container
4. **Default Methods** - Interface evolution
5. **Method References** - Shorthand for lambdas
6. **New Date/Time API** - java.time package
7. **Nashorn JavaScript Engine** - Better JS support
8. **CompletableFuture** - Asynchronous programming
9. **Parallel Array Sorting** - Arrays.parallelSort()
10. **Type Annotations** - Annotations anywhere

3. Generics in Java

Generics enable types (classes/interfaces) to be parameters when defining classes, interfaces, and methods.

Benefits:

- Stronger type checks at compile time
- Elimination of casts
- Enabling algorithms to work on different types

Example:

```
java
Copy
// Generic class
class Box<T> {
    private T t;
    public void set(T t) { this.t = t; }
    public T get() { return t; }
}

// Usage
Box<Integer> integerBox = new Box<>();
```

```
integerBox.set(10);
Integer value = integerBox.get(); // No casting needed

// Generic method
public static <T> void printArray(T[] array) {
    for (T element : array) {
        System.out.println(element);
    }
}
```

4. Serialization and Deserialization

Serialization is the process of converting an object into a byte stream for storage or transmission.

Deserialization is the reverse process of reconstructing the object from the byte stream.

Example:

```
java
Copy
// Serializable class
class Person implements Serializable {
    private String name;
    private int age;
    // Constructors, getters, setters
}

// Serialization
Person p = new Person("John", 30);
try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("pe
rson.ser"))) {
    oos.writeObject(p);
}

// Deserialization
try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("perso
n.ser"))) {
    Person deserialized = (Person) ois.readObject();
}
```

5. Collection Comparisons

LinkedList vs ArrayList:

Feature	LinkedList	ArrayList
Underlying	Doubly-linked list	Dynamic array
Insert/Delete	O(1) at ends, O(n) middle	O(n)
Access	O(n)	O(1)
Memory	More overhead (node links)	Less overhead
Use Cases	Frequent insertions/deletions	Random access, iteration

HashMap vs Hashtable:

Feature	HashMap	Hashtable
Synchronization	Not synchronized	Synchronized
Null Keys	Allows one null key	Doesn't allow null keys
Performance	Faster	Slower due to sync
Iteration	Fail-fast iterator	Not fail-fast
Legacy	Newer (Java 1.2)	Older (Java 1.0)

6. Design Patterns

Design patterns are reusable solutions to common software design problems.

Categories:

1. **Creational** (object creation):
   - Singleton, Factory, Abstract Factory, Builder, Prototype
2. **Structural** (object composition):
   - Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy
3. **Behavioral** (object interaction):
   - Observer, Strategy, Command, State, Template Method, Visitor, Chain of Responsibility

Singleton Example:

```
java
Copy
public class Singleton {
    private static Singleton instance;

    private Singleton() {}

    public static synchronized Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

Microservices Architecture

1. Circuit Breaker Pattern

The circuit breaker pattern prevents a system from making repeated failed requests to a failing service.

States:

1. **Closed**: Requests pass through normally
2. **Open**: Requests fail immediately without attempting
3. **Half-Open**: Limited requests allowed to test recovery

Implementation with Resilience4j:

```
java
Copy
CircuitBreaker circuitBreaker = CircuitBreaker.ofDefaults("backendService");

Supplier<String> decoratedSupplier = CircuitBreaker
    .decorateSupplier(circuitBreaker, backendService::doSomething);

Try.ofSupplier(decoratedSupplier)
    .recover(throwable -> "Fallback response");
```

2. Fault Tolerance in Microservices

Strategies for fault tolerance:

1. **Retries**: Transient failures
2. **Timeouts**: Prevent hanging calls
3. **Fallbacks**: Alternative responses
4. **Bulkheads**: Isolate failures
5. **Circuit Breakers**: Stop cascading failures
6. **Rate Limiters**: Prevent overload

3. Maintaining Consistency

Approaches for microservices consistency:

1. **Saga Pattern**: Distributed transactions using local transactions and compensating actions
2. **Event Sourcing**: Store state changes as events
3. **CQRS**: Separate read and write models
4. **Outbox Pattern**: Reliable event publishing
5. **Two-Phase Commit**: For some cases (limited use)

4. API Gateway

An API Gateway is a single entry point that:

- Routes requests to appropriate services
- Aggregates responses
- Handles cross-cutting concerns:
  - Authentication/authorization
  - Rate limiting
  - Load balancing
  - Caching
  - Request/response transformation

5. Monolithic vs Microservices

Aspect	Monolithic	Microservices
Architecture	Single codebase	Loosely coupled services
Scaling	Vertical	Horizontal
Deployment	Single unit	Independent
Technology	Limited to one stack	Polyglot
Complexity	Simpler initially	Complex coordination
Database	Shared	Dedicated per service
Fault Isolation	Poor	Good

6. API Versioning Methods

Common API versioning approaches:

1. **URI Path**: `/api/v1/resource`
2. **Query Parameter**: `/api/resource?version=1`
3. **Header Versioning**: `Accept: application/vnd.company.api.v1+json`
4. **Media Type**: `Content-Type: application/vnd.company.v1+json`
5. **Hostname**: `v1.api.company.com`

Spring & Spring Boot

1. Spring Annotations

Core Spring annotations:

1. **@Component**: Marks a class as Spring bean
2. **@Service**: Specialized @Component for service layer
3. **@Repository**: Specialized @Component for DAOs
4. **@Autowired**: Dependency injection
5. **@Configuration**: Configuration class

6. **@Bean**: Method-level bean definition
7. **@RequestMapping**: Maps web requests
8. **@SpringBootApplication**: Main application class
9. **@Value**: Injects property values
10. **@Profile**: Conditional bean registration

2. Spring Actuator

Spring Boot Actuator provides production-ready features:

- Health checks (`/actuator/health`)
- Metrics (`/actuator/metrics`)
- Environment details (`/actuator/env`)
- Thread dump (`/actuator/threaddump`)
- Custom endpoints

Configuration:

```
properties
Copy
management.endpoints.web.exposure.include=health,info,metrics
management.endpoint.health.show-details=always
```

3. Configuration in Spring

Spring supports multiple configuration approaches:

1. **application.properties/application.yml**

```
properties

Copy

app.name=MyApp
app.description=${app.name} is a Spring Boot application
```

2. **@Configuration classes**

```
java

Copy
```

```
@Configuration
public class AppConfig {
    @Bean
    public MyService myService() {
        return new MyServiceImpl();
    }
}
```

3. **Profile-specific properties**

- `application-dev.properties`
- `application-prod.properties`

4. **@PropertySource**

```
java

Copy

@Configuration
@PropertySource("classpath:custom.properties")
public class CustomConfig {}
```

4. @Controller vs @RestController

Feature	@Controller	@RestController
Usage	Traditional MVC	REST APIs
Response	Requires @ResponseBody	Automatic @ResponseBody
View Resolution	Yes	No
Content-Type	Typically HTML	Typically JSON/XML

Example:

```
java
Copy
@Controller
public class TraditionalController {
    @GetMapping("/greet")
    public String greet(Model model) {
        model.addAttribute("message", "Hello");
        return "greetView"; // Resolves to greetView.html
    }
```

```
}

@RestController
public class ApiController {
    @GetMapping("/api/greet")
    public Greeting greet() {
        return new Greeting("Hello"); // Returns JSON
    }
}
```

5. Database Connection in Spring

Steps to connect:

1. Add dependency (spring-boot-starter-data-jpa)
2. Configure properties:

```
properties

Copy

spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=user
spring.datasource.password=pass
spring.jpa.hibernate.ddl-auto=update
```

3. Create Entity classes
4. Create Repository interfaces

Multiple Databases:

```
java
Copy
@Configuration
@EnableJpaRepositories(
    basePackages = "com.app.db1.repositories",
    entityManagerFactoryRef = "db1EntityManager",
    transactionManagerRef = "db1TransactionManager"
)
public class Db1Config {

    @Bean
    @ConfigurationProperties("spring.db1.datasource")
```

```
    public DataSource db1DataSource() {
        return DataSourceBuilder.create().build();
    }

    @Bean
    public LocalContainerEntityManagerFactoryBean db1EntityManager() {
        // Configure with db1DataSource
    }

    @Bean
    public PlatformTransactionManager db1TransactionManager() {
        // Configure with db1EntityManager
    }
}
```

6. Caching in Spring Boot

Implementation Steps:

1. Add dependency: `spring-boot-starter-cache`
2. Enable caching: `@EnableCaching` on main class
3. Annotate methods:

```
java

Copy

@Cacheable("products")
public Product getProductById(Long id) {
    // DB call
}

@CacheEvict(value="products", key="#id")
public void updateProduct(Product product) {
    // Update DB
}
```

4. Configure cache provider (EhCache, Redis, etc.)

7. Property Files in Spring Boot

Common property files:

1. **application.properties** - Main configuration
2. **application.yml** - YAML alternative
3. **application-{profile}.properties** - Profile-specific
4. **bootstrap.properties** - For Spring Cloud Config
5. **Custom property files** - Loaded via @PropertySource

8. MVC Architecture

Model-View-Controller Pattern:

1. **Model**:
   - Represents application data and business logic
   - Typically POJOs or entities
2. **View**:
   - Presentation layer (HTML, JSP, Thymeleaf, etc.)
   - Displays model data
3. **Controller**:
   - Handles user requests
   - Processes input
   - Updates model
   - Returns appropriate view

Spring MVC Flow:

1. DispatcherServlet receives request
2. HandlerMapping finds appropriate controller
3. Controller processes request, returns ModelAndView
4. ViewResolver resolves view name to actual view
5. View renders response using model data

Example:

```java
```

Copy

```
@Controller
public class ProductController {

    @Autowired
    private ProductService service;

    @GetMapping("/products")
    public String listProducts(Model model) {
        model.addAttribute("products", service.getAllProducts());
        return "products/list";
    }

    @PostMapping("/products")
    public String addProduct(@ModelAttribute Product product) {
        service.saveProduct(product);
        return "redirect:/products";
    }
}
```